AI Development Documentation

1. Inputs Provided to ChatGPT

The AI session was initiated by providing the full assignment prompt for Part V of CS 3773, which outlined all deliverables and requirements. From there, the following inputs were supplied across the session:

Initial Design Package

- Full assignment description specifying at least 3 use cases, alternative/error flow, GitHub submission, and 10-minute demo
- Textual requirements including actor definitions (Student, Staff) and a list of 9 functional requirements covering search, reserve, cancel, check-in, availability checking, error messaging, and validation
- Use case descriptions for: Search Study Rooms, Reserve Study Room, and Cancel Reservation
- Data model with three entities: Student (studentID, name, email), StudyRoom (roomID, building, capacity, features, availability), and Reservation (reservationID, studentID, roomID, date, startTime, endTime, status)
- GUI screen descriptions including a search/filter panel, results page, confirmation page, and error state
- Previously completed UML diagrams uploaded as draw.io files (UseCase.drawio, ClassDiagram.drawio, SequenceDiagram.drawio, stateDiagram.drawio)
- A screenshot of the real UTSA Library room reservation interface as a visual reference for UI layout and style

Iterative Correction Prompts

- Explicit request to add a login screen with name and email fields
- Request to fix room capacity filtering (rooms with capacity greater than 2 were not appearing)
- Request to add more room cards with placeholder images to populate the results view
- Request to add assistive features: large text toggle, high contrast toggle, and language selection
- Request to move the reservation confirmation message to the top of the page after booking
- Request to make the language selector actually change visible interface text rather than only showing an alert
- Request to add comprehensive comments to all three code files (index.html, style.css, script.js)

- Requests for demo script, reflection answers, and README content

2. What ChatGPT Understood Well

The AI demonstrated strong understanding of several aspects of the project from early in the session:

Project Scope and MVP Structure

- Immediately identified the correct three core use cases to implement: search rooms, reserve a room, and cancel a reservation
- Understood that the prototype should be a frontend-only MVP using HTML, CSS, and JavaScript with no backend or database required
- Correctly recommended against adding login systems, admin dashboards, or real database connections in early iterations, keeping the scope appropriate

Code Generation

- Generated functional HTML/CSS/JavaScript code that correctly implemented room search with filter logic, reservation confirmation, and cancellation
- Correctly structured the JavaScript filter to match rooms by capacity and boolean features (whiteboard, power outlets, accessibility)
- When asked to add a login screen, correctly implemented show/hide logic using display:none and display:flex
- Generated well-formatted room cards with placeholder images, feature tags, and a Book Now button matching the style of the UTSA reference screenshot

UML Feedback and Correction

- Accurately interpreted the professor's written feedback on all four UML diagrams and translated it into specific, actionable draw.io instructions
- Correctly explained the difference between include and extend relationships in use case diagrams, including the direction arrows should point
- Understood that the class diagram was missing MVC components and provided a complete ReservationController class with correct method signatures, parameter types, and return types
- Correctly explained UML sequence diagram conventions including object lifelines, activation bars, dashed return arrows, and the colon naming convention for objects

3. What ChatGPT Misunderstood or Ignored

Login Page Omitted in First Output

The initial code generation did not include a login screen despite login being one of the use cases in the design package. The login page had to be explicitly requested as a separate follow-up prompt. This was a meaningful omission since Login was marked as an included dependency of every other use case in the use case diagram

Room Capacity Filtering Bug

The first version of the search function used a basic string comparison for capacity rather than a proper integer comparison. This caused rooms with a capacity greater than 2 to not appear when a user searched for a capacity of 2 or more. The bug required a separate correction prompt to fix by converting the input to an integer using parseInt() and defaulting to 0 when the field was empty.

Assistive Features MIssing from First Version

The initial prototype did not include any assistive features despite them being part of the use case diagram (Use Assistive Features connected via extend to Search Rooms, and Change Language as a separate extend relationship). These were entirely absent until explicitly requested, which meant the AI treated them as optional rather than reading them as requirements from the provided design materials.

Superficial Language Feature

When the language dropdown was first added, selecting a language only triggered a browser alert dialog rather than actually changing any visible text on the page. This was pointed out as a problem and corrected, but the initial implementation ignored the functional intent of the feature entirely.

Duplicate HTML Elements from Incremental Edits

Because the AI was providing incremental code patches rather than complete file replacements in some responses, combining the patches resulted in a duplicate header element and a double nested .main div in the HTML file. These structural errors required a cleanup pass and a complete corrected version to be provided.

Confirmation Box Placement

After a room was booked, the confirmation message was initially rendered below the room results cards rather than at the top of the content area. This had to be corrected by reordering the HTML elements and adding a window.scrollTo() call in the JavaScript.

4. Challenges Encountered

Communicating Requirements

The most consistent challenge throughout the session was that the AI required very specific and often simplified instructions to produce the correct output. Vague or high-level requests (such as 'make it look like the UTSA example') were interpreted loosely, while detailed step-by-step descriptions produced more accurate results. Requirements that seemed obvious from the context, such as including a login page when login was listed as a use case, were still missed without explicit prompting.

Incremental Patching Caused Integration Errors

Because the AI frequently provided code snippets or partial updates rather than complete file replacements, combining those patches manually introduced bugs. The duplicate header and nested div issues described above were direct consequences of this. Future iterations should request full file replacements at each step rather than partial patches.

UML Diagram Iteration in draw.io

A rough UML was generated by AI and then manually translated to draw.io diagrams. This required significant effort.

Verification and Testing

The generated code required multiple rounds of testing in the browser to identify issues that were not apparent from reading the code alone. Capacity filtering, language switching, and the confirmation message placement all appeared correct in the code snippets but only revealed problems when the application was actually run. Running and testing AI-generated code rather than assuming it works as described is important.

Precisely Outlining Scope

In early responses the AI tended to suggest or add features beyond what was requested, such as recommending EC2 deployment, PHP backends, or database

integration. These suggestions had to be actively redirected to keep the prototype simple and aligned with the MVP requirements.

5. How Prompts Were Revised

Breaking Down Large Requests

Rather than asking the AI to generate the entire application at once, the session moved to smaller, incremental requests. For example, instead of asking for 'the full app with all features,' each feature was requested separately: first the base search and reserve functionality, then login, then assistive features. This reduced the chance of the AI omitting elements.

Referencing Specific Use Cases by Name

When features were missing, prompts were revised to explicitly name the use case or requirement being referenced. For example, rather than saying 'add accessibility,' the revised prompt said 'add the assistive features from the use case diagram, specifically large text, high contrast, and language selection.' This gave the AI a concrete anchor to work from.

Providing Visual references

Providing a screenshot of the real UTSA Library reservation interface and our own manually-created GUI diagram dramatically improved the quality of the UI output. The AI moved from a basic centered layout to a two-column design with a filter panel, room cards, placeholder images, and color-coded tags that matched the reference image. Visual references proved more effective than textual layout descriptions.

Requesting Complete File Replacements

After the HTML nesting bug was discovered, subsequent code requests explicitly asked for the complete corrected file rather than a patch. This prevented integration errors from combining partial updates.

Specifying What Not to Include

Early prompts were revised to include explicit exclusions, such as 'do not add a login system, admin dashboard, or database.' This helped keep the AI focused on the MVP scope rather than expanding the feature set beyond what was required.